# راهنمای جامع مرجع STM32H7 + TouchGFX + FreeRTOS

**تحلیل تخصصی معماری MVP، مدیریت صف‌های CMSIS-RTOS v2، پیکربندی LTDC/SDRAM و تحلیل کامل کدها**

### مقدمه و هدف این مستند مرجع

این سند، راهنمای کامل و استاندارد صنعتی جهت پیاده‌سازی سامانه‌های گرافیکی پیشرفته بر روی میکروکنترلرهای سری STM32H7 با استفاده از فریم‌ورک TouchGFX و سیستم‌عامل FreeRTOS می‌باشد. در این مستند، از یک نمونه کاربردی (قرائت کلید فیزیکی KEY1 متصل به پایه PH3 و تغییر متن پویا روی LCD) استفاده شده تا تمامی لایه‌های سخت‌افزاری، سیستم‌عامل، لایه ارتباطی و معماری نرم‌افزاری MVP به تفصیل شکافته شوند. هدف اصلی، ارائه یک الگوی توسعه است که مهندس نرم‌افزار بتواند هرگونه داده سخت‌افزاری (CAN، UART، ADC، حسگرها) را بدون خطر Data Race یا فریز شدن رابط کاربری، به صفحه نمایش منتقل کند.

## فهرست مطالب تخصصی مستند


1. **فصل اول:** تشریح کامل معماری MVP (Model-View-Presenter) و تحلیل چرخه حیات (Lifecycle)
2. **فصل دوم:** مسئله چندتسکی (Multi-threading)، عدم Thread-Safety در TouchGFX و ضرورت FreeRTOS Queue
3. **فصل سوم:** پیکربندی گام‌به‌گام STM32CubeMX، حافظه‌های SDRAM/LTDC و تولید کد با CMSIS-RTOS v2
4. **فصل چهارم:** تحلیل دقیق کد تسک ورودی StartButtonTask، تابع HAL_GPIO_ReadPin و دیباونس نرم‌افزاری
5. **فصل پنجم:** تحلیل عمیق کلاس‌های Model.hpp و Model.cpp و مدیریت حلقه osMessageQueueGet
6. **فصل ششم:** پیاده‌سازی لایه واسط ModelListener و الگوی ارتباطی double-binding با Presenter
7. **فصل هفتم:** مدیریت ویجت‌ها، حافظه Wildcard Buffer، فونت فارسی/انگلیسی و توابع کلاس Unicode
8. **فصل هشتم:** جریان کامل داده‌ها (End-to-End Data Flow)، جدول عیب‌یابی جامع و خطاهای متداول پروژه
9. **فصل نهم:** الگوی تعمیم ساخت یافته (Generalization Pattern) برای انواع داده‌های آنالوگ و دیجیتال
10. **فصل دهم:** چک‌لیست عیب‌یابی سریع، آنالیز پشته/حافظه (Heap & Stack) و مراجع استاندارد


**نمای کلی مسیر جریان داده در معماری MVP + RTOS**

```
[ Hardware KEY1 ] ---> (Button Task - FreeRTOS) ---> [ osMessageQueue (FIFO) ]
                                                              |
                                                              v
[ TouchGFX View ] <--- [ Presenter ] <--- [ ModelListener ] <--- (Model::tick in GUI Task)
```

## فصل ۱: منطق معماری MVP و چرایی الزام آن در پروژه‌های صنعتی

معماری **Model-View-Presenter (MVP)** الگوی طراحی استاندارد شرکت STMicroelectronics برای جداسازی منطق برنامه (Business Logic) از لایه بصری (User Interface) است. بدون استفاده از الگوی MVP، توسعه‌دهنده مجبور است کد خواندن پایه GPIO یا پروتکل‌های ارتباطی را مستقیماً درون کلاس‌های گرافیکی بنویسد که باعث بروز مشکلات زیر می‌شود:

- **عدم امکان نگهداری و توسعه:** تغییر در چیدمان گرافیکی باعث شکستن منطق سیستم می‌شود.
- **وابستگی سخت‌افزاری:** جهت تست منطق برنامه حتماً باید سخت‌افزار واقعی و LCD متصل باشد و امکان تست در Simulator از بین می‌رود.
- **پیچیدگی کد:** کلاس‌های گرافیکی بسیار سنگین و غیرقابل دیباگ می‌شوند.

### تفکیک مسئولیت‌ها در الگوی MVP

| نقش | کلاس‌های مرتبط | مسئولیت‌ها و تعاریف |
|---|---|---|
| **Model** | `Model.hpp / Model.cpp` | مخزن داده‌ها و منطق برنامه. ارتباط با دنیای RTOS و سخت‌افزار را بر عهده دارد. هیچ اطلاعاتی از نحوه نمایش و ویجت‌های روی صفحه ندارد. |
| **View** | `screenView.cpp / screenViewBase.cpp` | مدیریت مستقیم ویجت‌های گرافیکی (Text, Image, Button). هیچ منطق تصمیمی ندارد و صرفاً دستورات نمایش را از Presenter می‌گیرد. |
| **Presenter** | `screenPresenter.cpp / ModelListener.hpp` | واسط میان Model و View. تغییرات Model را دریافت کرده و به View فرمان بروزرسانی می‌دهد. همچنین رویدادهای لمسی View را به Model منتقل می‌سازد. |

## قوانین طلایی ارتباطات در MVP

1. **ارتباط View و Presenter:** کلاس View از طریق اشاره‌گر `presenter` به Presenter متصل است. View هرگز مستقیماً Model را نمی‌شناسد.
2. **ارتباط Presenter و Model:** کلاس Presenter دارای یک اشاره‌گر به Model و همچنین وارث کلاس `ModelListener` است.
3. **وارونگی وابستگی (Dependency Inversion):** کلاس Model کلاس `screenPresenter` را مستقیماً نمی‌شناسد! تنها یک اشاره‌گر از نوع `ModelListener*` نگه می‌دارد. این کار اجازه می‌دهد با تعویض صفحه نمایش (Screen)، Presenter همان صفحه به Model متصل شود.

> **مفهوم الگوی چرخه حیات (Lifecycle) در TouchGFX**
>
> هنگام تغییر صفحه نمایش، فریمورک TouchGFX شیء View و Presenter صفحه قبلی را نابود کرده و اشیاء صفحه جدید را روی حافظه استاتیک (و نه Heap) بازسازی می‌کند. زنجیره متدهای فراخوانی شده در تغییر صفحه به شرح زیر است:
>
> ```
> Screen Change Sequence: 1. currentPresenter.deactivate() 2.
> currentView.tearDownScreen() 3. newView.setupScreen() 4. newPresenter.bind(newModel)
> 5. newModel.bind(newPresenter) 6. newPresenter.activate()
> ```

## فصل ۲ : مسئله چندتسکی و چرا صف FreeRTOS الزام قطعی است؟

در پروژه‌های STM32H7 همراه با سیستم‌عامل FreeRTOS، معمولاً چندین تسک با اولویت‌های مختلف به طور همزمان در حال اجرا هستند. جدول زیر ساختار تسک‌های اصلی در این پروژه را نشان می‌دهد:

| نام تسک | اولویت (Priority) | حجم پشته (Stack Size) | وظیفه اصلی |
|---|---|---|---|
| `defaultTask` | osPriorityNormal | 128 Words (512 Bytes) | تسک پیش‌فرض سیستم‌عامل (آماده برای عملیات ثانویه) |
| `GUI_Task` | osPriorityLow | 2048 Words (8192 Bytes) | اجرای موتور TouchGFX، رندر گرافیکی و فراخوانی متد Model::tick |
| `ButtonTask` | osPriorityLow / Normal | 128 Words (512 Bytes) | پایش وضعیت کلید سخت‌افزاری KEY1 (PH3) با متد Polling/Debounce |

### علل ممنوعیت تغییر مستقیم UI از داخل ButtonTask

**خطرات عدم رعایت Thread-Safety در TouchGFX**

1. **عدم وجود Thread-Safety:** فریمورک TouchGFX دارای سیستم Sync داخلی برای تغییر متغیرهای ویجت‌ها از تسک‌های دیگر نیست. فراخوانی مستقیم توابع View از ButtonTask منجر به Corrupt شدن بافر گرافیکی می‌شود.
2. **مسئله Race Condition:** اگر در زمانی که تسک GUI در حال رندر فریم جدید روی SDRAM است، تسک ButtonTask بافر متن را تغییر دهد، HardFault یا بهم‌ریختگی شدید بصری ایجاد می‌گردد.
3. **تخریب اشاره‌گرها در طول Screen Transition:** اگر تسک سخت‌افزاری یک اشاره‌گر از View نگه دارد و کاربر صفحه را عوض کند، آن View نابود شده و دسترسی به آن منجر به Crash کامل میکروکنترلر می‌شود.

## حل مشکل با مکانیزم Message Queue در CMSIS-RTOS v2

صف (Queue) یک بافر FIFO (First-In, First-Out) نخامن (Thread-safe) است که توسط سیستم‌عامل مدیریت می‌شود. تسک سخت‌افزار وضعیت جدید را `Put` کرده و تسک GUI در متد `Model::tick()` آن را `Get` می‌کند. هیچ بلاک شدنی برای تسک GUI رخ نمی‌دهد زیرا Timeout برابر صفر قرار داده می‌شود.

**معماری انتقال داده بین Taskها با Message Queue**

```
[ButtonTask] --(osMessageQueuePut)--> [QueueButtonHandle (Size: 16)] --(osMessageQueueGet)--
> [Model::tick() in GUI_Task]
```

## فصل ۳: پیکربندی STM32CubeMX و کدهای تولید شده سیستم‌عامل

برای راه‌اندازی صحیح صف پیام و تسک‌ها در محیط STM32CubeMX، مراحل زیر باید با دقت اعمال شوند تا تضاد در اندازه‌های متغیرها و تخصیص حافظه پیش نیاید:

### مراحل تنظیمات در STM32CubeMX

1. در بخش **Middleware -> FREERTOS**، واسط را روی CMSIS_V2 قرار دهید.
2. به تب **Tasks and Queues** رفته و در بخش Queues یک صف جدید بسازید:
   - **Queue Name:** QueueButton
   - **Queue Size:** 16 (امکان ذخیره ۱۶ پیام متوالی)
   - **Item Size:** uint32_t یا sizeof(uint32_t) (۴ بایت)
   - **Allocation:** Dynamic
3. در همان تب، یک تسک جدید اضافه کنید:
   - **Task Name:** ButtonTask
   - **Priority:** osPriorityLow یا osPriorityNormal
   - **Stack Size:** 128 Words
   - **Entry Function:** StartButtonTask
4. پایه مربوط به کلید را (مثلا PH3) روی GPIO_Input قرار داده و نام لیبل آن را KEY1 بگذارید.

> **رفع خطای رایج عدم تطابق Item Size**
>
> در پروژه‌های پیشین، صف با اندازه sizeof(uint16_t) ساخته شده بود اما تسک متغیر uint32_t ارسال می‌کرد. این کار باعث کپی ناقص بایت‌ها (Data Truncation) می‌شد. همواره نوع متغیر ارسالی با Item Size تعریف شده در CubeMX باید کاملاً همخوانی داشته باشد.

## کد تولید شده در فایل main.c

پس از تولید کد، کدهای زیر در main.c جهت تعریف و مقداردهی صف و تسک ایجاد می‌شوند:

```
/* Global definitions in main.c */ osMessageQueueId_t QueueButtonHandle; const osMessageQueueAttr_t QueueButton_attributes = { .name = "QueueButton" }; /* Inside main() function before osKernelStart() */ osKernelInitialize(); /* Create the Queue for 16 items of uint32_t */ QueueButtonHandle = osMessageQueueNew(16, sizeof(uint32_t), &QueueButton_attributes); /* Create the Button Task */ const osThreadAttr_t ButtonTask_attributes = { .name = "ButtonTask", .stack_size = 128 * 4, .priority = (osPriority_t) osPriorityLow, }; ButtonTaskHandle = osThreadNew(StartButtonTask, NULL, &ButtonTask_attributes); osKernelStart();
```

## فصل ۴: تحلیل کامل کد تسک ورودی StartButtonTask

تسک سخت‌افزاری وظیفه دارد پایه مربوط به کلید را پایش کرده و تنها هنگام **تغییر وضعیت (State Change)**، مقدار جدید را به صف ارسال کند تا از ارسال داده‌های تکراری و پر شدن بیهوده صف جلوگیری شود.

### پیاده‌سازی بهینه تابع StartButtonTask در main.c

```
/* Task implementation in main.c or gpio.c */ void StartButtonTask(void *argument) { /*
Initialize last state to dummy value to ensure first event triggers update */ uint32_t
lastState = 0xFFFFFFFF; for(;;) { /* Read hardware pin state using HAL driver */ uint32_t
currentState = (uint32_t)HAL_GPIO_ReadPin(KEY1_GPIO_Port, KEY1_Pin); /* Detect edge
change */ if (currentState != lastState) { lastState = currentState; /* Put the new state
into the queue. Timeout = 0 (Non-blocking) */ osStatus_t status =
osMessageQueuePut(QueueButtonHandle, ¤tState, 0U, 0U); if (status != osOK) { /* Optional:
Handle queue full error or warning */ } } /* 20ms delay provides automatic hardware
debouncing */ osDelay(20); } }
```

## تحلیل توابع کلیدی و پارامترها

| تابع / پارامتر | پارامترها / مقادیر | توضیح فنی و کارکرد |
|---|---|---|
| HAL_GPIO_ReadPin | KEY1_GPIO_Port, KEY1_Pin | خواندن وضعیت فیزیکی پایه پایه PH3. مقدار بازگشتی GPIO_PIN_SET (1) یا GPIO_PIN_RESET (0) است. |
| osMessageQueuePut | mq_id, msg_ptr, msg_prio, timeout | ارسال داده به صف. مقدار Timeout برابر 0 قرار داده شده تا در صورت پر بودن صف، تسک معطل نشود. |
| osDelay(20) | ticks (20 ms) 20 | تسک را به مدت ۲۰ میلی‌ثانیه به حالت Blocked می‌برد. این وقفه دو اثر دارد: ۱. آزاد کردن CPU برای GUI Task. ۲. دیباونس نرم‌افزاری نویز مکانیکی کلید. |

**چرا ارسال فقط هنگام تغییر وضعیت بهتر از ارسال مداوم است؟**

اگر شرط currentState != lastState حذف شود، در هر ثانیه ۵۰ پیام تکراری وارد صف می‌شود. این امر باعث اشغال پهنای باند پردازشی در متد Model::tick() می‌گردد. با ارسال تغییرات (Event-driven)، مصرف پردازنده به حداقل می‌رسد.

## فصل ۵: تحلیل عمیق کلاس‌های Model.hpp و Model.cpp

کلاس Model قلب سیستم ارتباطی در لایه GUI است. این کلاس در فریم‌ورک TouchGFX به صورت Singleton مدیریت شده و متد `tick()` آن در هر فریم گرافیکی (حدوداً ۶۰ بار در ثانیه یا هر ۱۶.۶ میلی‌ثانیه) صدا زده می‌شود.

### فایل ساختاری Model.hpp

```
#ifndef MODEL_HPP #define MODEL_HPP class ModelListener; class Model { public: Model(); /* Binds active Presenter's ModelListener interface to Model */ void bind(ModelListener* listener) { modelListener = listener; } /* Called automatically by TouchGFX framework once per frame (16.6ms) */ void tick(); protected: ModelListener* modelListener; }; #endif // MODEL_HPP
```

### فایل پیاده‌سازی Model.cpp (سازگار با Simulator و Target)

```
#include <gui/model/Model.hpp> #include <gui/model/ModelListener.hpp> #ifndef SIMULATOR #include "cmsis_os2.h" #include "main.h" /* Reference to global queue handle declared in main.c */ extern "C" osMessageQueueId_t QueueButtonHandle; #endif Model::Model() : modelListener(0) { } void Model::tick() { #ifndef SIMULATOR uint32_t rxState; /* Flush all pending messages from the queue in current frame */ while (osMessageQueueGet(QueueButtonHandle, &rxState, NULL, 0U) == osOK) { /* Active Low / Active High Logic resolution */ bool isPressed = (rxState == 1); // Adjust depending on Pull-Up/Down if (modelListener != 0) { modelListener->notifyButtonChanged(isPressed); } } #endif }
```

**نکات حیاتی در کد Model.cpp**

- **استفاده از `extern "C"`:** چون متغیر QueueButtonHandle در main.c (زبان C) تعریف شده، باید درون کد C++ با `extern "C"` اعلان شود تا دچار Name Mangling نگردد.
- **حلقه `while` به جای `if`:** اگر چند پیام در طول یک فریم وارد صف شده باشد، حلقه `while` تمامی پیام‌ها را پردازش کرده و صف را خالی می‌کند تا از تاخیر (Lag) جلوگیری شود.

## فصل ۶: پیاده‌سازی لایه واسط ModelListener و Presenter

برای برقراری ارتباط دوطرفه و بدون وابستگی مستقیم، کلاس `ModelListener` به عنوان یک کلاس پایه انتزاعی (Abstract Base Class) تعریف می‌شود.

### فایل ModelListener.hpp

```
#ifndef MODELLISTENER_HPP #define MODELLISTENER_HPP #include <gui/model/Model.hpp> class ModelListener { public: ModelListener() : model(0) {} virtual ~ModelListener() {} void bind(Model* m) { model = m; } /* Virtual callback functions to be overridden by active Presenter */ virtual void notifyButtonChanged(bool buttonPressed) {} protected: Model* model; }; #endif // MODELLISTENER_HPP
```

## کلاس screenPresenter: اتصال دهنده Model به View

کلاس Presenter صفحه، هم از `touchgfx::Presenter` و هم از `ModelListener` ارث‌بری می‌کند. این کلاس رویدادهای دریافتی از Model را به متدهای View منتقل می‌کند.

### فایل screenPresenter.hpp

```
#ifndef SCREENPRESENTER_HPP #define SCREENPRESENTER_HPP #include
<gui/model/ModelListener.hpp> #include <mvp/Presenter.hpp> class screenView; class
screenPresenter : public touchgfx::Presenter, public ModelListener { public:
screenPresenter(screenView& v); virtual void activate(); virtual void deactivate();
virtual ~screenPresenter() {} /* Override callback from ModelListener */ virtual void
notifyButtonChanged(bool pressed) override; private: screenPresenter(); screenView& view;
}; #endif
```

### فایل screenPresenter.cpp

```
#include <gui/screen_screen/screenPresenter.hpp> #include
<gui/screen_screen/screenView.hpp> screenPresenter::screenPresenter(screenView& v) :
view(v) { } void screenPresenter::activate() { } void screenPresenter::deactivate() { }
void screenPresenter::notifyButtonChanged(bool pressed) { /* Forward data directly to
View */ view.updateButtonState(pressed); }
```

## فصل ۷: پیاده‌سازی View، حافظه Wildcard و توابع Unicode

کلاس screenView مسئول مستقیم بروزرسانی عناصر بصری روی صفحه نمایش است. در TouchGFX، برای نمایش رشته‌های متغیر پویا از ویجت **Wildcard Text** استفاده می‌شود.

### تعریف متدها در screenView.hpp و screenView.cpp

```
/* In screenView.hpp */ #ifndef SCREENVIEW_HPP #define SCREENVIEW_HPP #include
<gui_generated/screen_screen/screenViewBase.hpp> #include
<gui/screen_screen/screenPresenter.hpp> class screenView : public screenViewBase {
public: screenView(); virtual ~screenView() {} virtual void setupScreen(); virtual void
tearDownScreen(); /* Custom method to update UI */ void updateButtonState(bool pressed);
}; #endif
```

```
/* In screenView.cpp */ #include <gui/screen_screen/screenView.hpp> #include
<touchgfx/Unicode.hpp> screenView::screenView() { } void screenView::setupScreen() {
screenViewBase::setupScreen(); } void screenView::tearDownScreen() {
screenViewBase::tearDownScreen(); } void screenView::updateButtonState(bool pressed) { if
(pressed) { /* Safe copy ASCII to Unicode Buffer */
touchgfx::Unicode::strncpy(textArea1Buffer, "Pressed", TEXTAREA1_SIZE); } else {
touchgfx::Unicode::strncpy(textArea1Buffer, "Not Pressed", TEXTAREA1_SIZE); } /*
Invalidate widget area to trigger redraw on next frame */ textArea1.invalidate(); }
```

## جدول توابع پرکاربرد کلاس Unicode در TouchGFX

| تابع | آرگومان‌ها | کاربرد و توضیح |
|---|---|---|
| `Unicode::strncpy` | `(dst, src, maxchars)` | کپی کردن یک رشته ASCII C-Style درون بافر UnicodeChar ویجت. |
| `Unicode::snprintf` | `dst, maxchars,) (... ,format` | قالب‌بندی اعداد و رشته‌ها (مانند sprintf) برای قرار دادن در Wildcard. |
| `()widget.invalidate` | بدون آرگومان | علامت‌گذاری مستطیل ویجت به عنوان Dirty جهت رندر مجدد در LTDC buffer. |

## فصل ۸: جریان کامل داده‌ها و جدول عیب‌یابی جامع پروژه

در جدول زیر تمامی خطاهای رایج که در طول پیاده‌سازی سیستم‌های TouchGFX + FreeRTOS رخ می‌دهند همراه با علل ریشه‌ای و راهکارهای اصلاحی گردآوری شده‌اند:

| نشانه و علامت خطا | علت ریشه‌ای (Root Cause) | راهکار اصلاحی و نحوه حل |
|---|---|---|
| تغییر کلید روی LCD اعمال نمی‌شود (بدون خطای کامپایل) | غلط املایی در نام تابع ModelListener (مثلاً عدم نوشتن حرف f یا عدم استفاده از override) | استفاده از دستور override در تعریف تابع Presenter تا کامپایلر تفاوت امضا را خطا دهد. |
| صفحه نمایش فریز می‌شود یا HardFault رخ می‌دهد | استفاده از osWaitForever در متد Model::tick() یا فراخوانی مستقیم View از تسک سخت‌افزار | تنظیم Timeout روی صفر (0U) در osMessageQueueGet و انتقال کلیه عملیات گرافیکی به GUI Task. |
| متن پویا به صورت متقاطع یا بریده بریده نمایش داده می‌شود | کوچک بودن Buffer Size در TouchGFX Designer یا عدم لایه‌بندی فونت ASCII/Unicode | افزایش Wildcard Buffer Size به میزان (حداکثر طول کاراکتر + ۱) و resize کردن ویجت. |
| خطای لینکر undefined reference to QueueButtonHandle | عدم استفاده از "extern "C هنگام include کردن فایل C در کد C++ | قرار دادن اعلان "extern "C osMessageQueueId_t QueueButtonHandle; درون بلوک هایدر Model.cpp. |
| مقدار بافر تغییر می‌کند ولی LCD بروز نمی‌شود | فراموش کردن فراخوانی متد invalidate() پس از تغییر بافر | فراخوانی متد textArea1.invalidate() بلافاصله بعد از کپی رشته یا مقدار در بافر. |

## فصل ۹: الگوی تعمیم ساخت‌یافته (Generalization Pattern)

برای اضافه کردن هرگونه داده جدید (مانند ADC، دمای سنسور، پیام CAN یا UART)، فرمول ۸ گام زیر را به صورت استاندارد طی کنید:

```
Standard 8-Step Generalization Rule: 1. CubeMX: Define Queue (e.g. QueueAdcHandle, Size:
uint16_t) 2. Hardware Task: Read ADC -> osMessageQueuePut(QueueAdcHandle, &adcVal, 0, 0)
3. Model.cpp: extern "C" osMessageQueueId_t QueueAdcHandle; 4. Model::tick():
osMessageQueueGet(QueueAdcHandle, &val, NULL, 0) -> modelListener->notifyAdc(val) 5.
ModelListener.hpp: virtual void notifyAdc(uint16_t val) {} 6. Presenter: void
notifyAdc(uint16_t val) override { view.updateAdc(val); } 7. View: void
updateAdc(uint16_t val) { Unicode::snprintf(...); progress1.setValue(val);
progress1.invalidate(); } 8. TouchGFX Designer: Place Widget & Run!
```

## فصل ۱۰: چک‌لیست عیب‌یابی سریع، آنالیز پشته و مراجع

### چک‌لیست ۱۰ دقیقه‌ای دیباگ پروژه در محیط Real-Time

1. آیا صف ساخته شده است؟ بررسی کنید `QueueButtonHandle != NULL` باشد.
2. آیا `osMessageQueueGet` در متد `tick()` مقدار `osOK` برمی‌گرداند؟ (یک Breakpoint بگذارید).
3. آیا اشاره‌گر `modelListener` غیر تهی (Non-zero) است؟ (در صورت تهی بودن، bind انجام نشده است).
4. آیا اندازه پشته GUI Task کافی است؟ (توصیه: حداقل ۲۰۴۸ کلمه برابر با ۸ کیلوبایت).
5. آیا حافظه SDRAM برای Framebufferها در STM32CubeMX و FMC به درستی پیکربندی شده است؟

### پایش میزان مصرف پشته با تابع FreeRTOS

جهت اطمینان از عدم سرریز پشته (Stack Overflow) در تسک‌های GUI و ButtonTask، می‌توانید از تابع زیر استفاده کنید:

```
/* Check remaining stack space in words */ UBaseType_t highWaterMarkGUI =
uxTaskGetStackHighWaterMark(GUI_TaskHandle); UBaseType_t highWaterMarkBtn =
uxTaskGetStackHighWaterMark(ButtonTaskHandle);
```

## منابع و مراجع رسمی جهت مطالعه بیشتر

- **TouchGFX Documentation (STMicroelectronics):** بخش‌های *Model-View-Presenter Architecture* و *Backend Communication* در support.touchgfx.com
- **ARM CMSIS-RTOS v2 API Reference:** بخش‌های *Message Queue Functions* و *Thread Management*
- **FreeRTOS Official Reference Manual:** بخش *Inter-Task Communication & Queue Management*
- **STM32H7 Reference Manual (RM0433):** بخش‌های *FMC Controller, LTDC Display Controller, Dynamic Memory Setup*

### تاییدیه پایانی مستند فنی

این مستند کلیه لایه‌های نرم‌افزاری، سخت‌افزاری، سیستم‌عامل و الگوریتم‌های MVP را برای معماری STM32H7 پوشش داده و به عنوان راهنمای استاندارد تیم‌های توسعه سامانه‌های نهفته (Embedded Systems) قابل اتکا می‌باشد.